Milestone Two

Caleb Hadley

Faculty of Computing & Data Sciences

CDSDX-799: Data Science Capstone

Joshua Von Korff

August 2, 2026

Problem Statement / Description

This paper develops a framework for financial crime detection utilizing account and transaction features. Applied across the IBM Transactions for AML, Paycheck Protection Program (PPP), and the European Credit Card (ECC) Fraud Detection datasets, the framework identifies common challenges like class imbalance, high false-positive rates, and overfitting by evaluating a number of supervised and unsupervised classification models. The presented solutions provide data scientists and crime investigators with strategies to combat crime in their organizations to save money, ensure regulatory compliance, and maintain customer confidence.

Conclusions

Across all datasets and each model, I used undersampling due to extreme class imbalance, where a small count of positive class instances indicate financial crime. Without undersampling, models always predict the non-fraudulent class. Building models that generalize to the positive class requires detection of positive instances while reducing false positives. This balances detecting the largest amount of crime while reducing strain on investigators. I also implemented Z-score normalization since each model evaluated is sensitive to the feature scale.

K-nearest Neighbors (KNN) (Breadth)

I first evaluated a baseline KNN model on the ECC dataset by applying a 0.8/0.2 train test split to evaluate model performance. This model learned to always predict the negative class. To resolve this, I introduced undersampling to 4,920 negative and 492 positive instances. Next, I performed feature scaling and re-evaluated the model, which attained a 0.99 negative class f1 and 0.89 f1 (0.98 precision and 0.82 recall) on the positive class. This model generalizes significantly better. Finally, I performed hyperparameter tuning of the number of neighbors (1, 3, 5,7, 9), metric (minkowski and cosine), and the power parameter (Manhattan, Euclidean). The

highest performing model evaluated by the f1 used the cosine metric and 5 neighbors received 0.94 macro avg f1, matching the performance of the baseline model. The final model maintains 0.99 negative class f1 and 0.98 precision/0.81 recall on the positive class, meeting the required business use case of detecting a large percentage of financial crime without many false positives.

Using the same baseline model evaluation approach on the IBM dataset, we see the same class imbalance issues. I performed undersampling to 51,770 negative class instances and 5,177 positive class instances. Next, I performed feature scaling and reevaluated the KNN model, resulting in 0.97 negative class 0.67 positive class f1 (0.73 precision and 0.62 recall). This model generalizes significantly better to the positive class. Finally, I performed the hyperparameter tuning. The optimal model used the cosine metric and 9 neighbors, achieving 0.97 f1 on the negative class and 0.68 f1 on the positive class (0.74 precision and 0.62 recall). This model offers a slight improvement over the baseline parameters, decreasing false positive cases.

I applied the same baseline model evaluation approach on the PPP dataset, uncovering the same class imbalance issue. I implemented undersampling to 950 negative and 95 positive class instances and scaled the features, resulting in 0.97 negative class f1 and a lift of positive class f1 to 0.70 (0.67 precision and 0.74 recall). This model generalizes better to the positive class and identifies the majority of fraudulent PPP loans. After hyperparameter tuning, the optimal parameters are 9 neighbors, p=1, and the minkowski metric. This model lifts the negative class f1 to 0.98 and the positive class f1 to 0.77 (0.75 precision and 0.79 recall). This model both decreases the false positive rate and increases the true positive detection rate.

Gradient Boost (Breadth)

On the ECC dataset a baseline Gradient Boosting Classifier with default parameters achieves 1.0 negative class f1 and 0.27 positive class. Adding undersampling and feature scaling

increases positive class f1 to 0.50 and decreases negative class f1 to 0.9. As shown in Figure 1, the default learning rate of 0.1 was optimal, as the performance stabilizes to a maximum at 60 trees. Higher learning rates overfit as shown by the loss increase as more trees are added after a threshold of around 15. A lower learning rate of 0.01 requires more than 150 to converge. Based on these findings, increasing the number of trees beyond the 100 will not improve performance. I tuned the max depth from 2 to 8 and found in cross validation with a max depth=7 the model gets 0.99 f1 on the negative class and 0.91 on the positive class (0.93 precision and 0.88 recall). This model generalized the best by optimizing for using more complex trees.

I repeated the same experiment on the IBM dataset. The baseline model learned to always predict the negative class due to the significant class imbalance. After undersampling and feature scaling, the performance increased to 0.69 positive class f1 and 0.97 negative class f1. As we can see in Figure 2, high learning rates such as 0.5 and 1.0 encounter overfitting, shown through the drastic drop in loss followed by an increase. The default 0.1 rate continually decreases the loss until a plateau at 150 trees. A lower rate of 0.01 requires significantly more trees. I recommend using the default learning rate and increasing the number of trees to 150. Finally, in parameter tuning I found an optimal max depth of 3. The final performance on the test set was 0.97 negative class f1 and 0.69 positive class (0.75 precision and 0.64 recall). Increasing the breadth of domain rules used for classifying the positive class had the largest impact on performance.

On the PPP dataset the baseline model learned to always predict the negative class, indicating class imbalance. Adding in undersampling and scaling increases the positive class performance to 0.67 precision and 0.84 recall. Next, I plotted the learning rate as shown in Figure 3. High learning rates of 0.5 or 1.0 led to overfitting, as we can see with the sharp decrease in loss followed by a large increase. The default learning rate of 0.1 loss plateaus

around 100 trees, and the lower rate of 0.01 approaches the same loss at around 150 trees. I recommend using the default 0.1 rate and 100 tree count and tuning the max depth. The final model uses a max depth of 2 and lifts the precision of the positive class to 0.71 while dropping recall to 0.79. The final model identifies the majority of fraudulent loans while reducing the number of false positive cases as well by using a large number of shallow trees.

K-Means and Silhouette Score (Breadth)

On the ECC dataset I applied undersampling due to memory limitations when training KMeans and feature scaling. Next, I determined the optimal number of clusters (k) by plotting the model inertia against k and found an optimal value of 2 using the elbow method and silhouette score. With k=2 I trained the final K-Means model and calculated the distance between each point and its closest centroid. I defined the 95th percentile as an anomaly threshold, where any points outside of that threshold are outliers which signal potential crime. I created scatter plots of the points colored by the anomaly flag and their ground truth class value with the X and Y axis set as high correlatory features with the target such as V12, V14, and V17 determined through EDA. Using the plots I found a separation of the anomalies. In practice, this method allows us to examine anomalies without a large sample of labels to train supervised learning models on. Evaluating the results of the anomaly detection against the target, the negative class f1 score was 0.98 and the positive class f1 was 0.28 (0.17 precision and 0.84 recall). This technique identified a majority of fraudulent transactions with limitations on the precision.

For the IBM dataset I applied the same undersampling and scaling approach as a preprocessing step. Using the elbow method and silhouette score I found an optimal value of k=6. When evaluating the final K-Means model using k=6, I encountered issues with performance. This model had 0.93 f1 score on the negative class but only 0.1 f1 on the positive

class (0.14 precision and 0.08 recall). One challenge when using K-Means is that this dataset contains 4 numeric and 40 categorical one-hot encoded features. Since K-Means uses Euclidean distance, the cluster centers are not close to actual points in terms of the binary features and all categories can look the same distance apart regardless of the true relationships. To improve the results, the Gower distance will be evaluated using DBScan to handle the mix of feature types.

On the PPP dataset, I completed the preprocessing steps to undersample and standardize the dataset. Next, in the elbow method and silhouette score I determined an optimal k=4. Similar to the IBM dataset, the final K-Means model trained on this dataset did not identify a large number of fraudulent loans. The negative class f1-score was 0.97, while the positive class f1 was 0.0. I hypothesize that the Euclidean distance was also problematic in this dataset due to a large number of categorical variables which are one-hot encoded. To improve results I will evaluate the Gower metric with DBScan.

Density-based Spatial Clustering of Applications with Noise (DBSCAN) (Depth)

Across each dataset I leveraged DBScan rather than Hierarchical Agglomerative clustering. The goal of clustering in my project is to detect outliers with the purpose of identifying sparse financial crime events that vary from standard transactions. With this goal in mind, DBScan is a better fit since it explicitly identifies and groups noise points. Since these crime events are rare, DBScan can flag these as noise while grouping typical points into clusters.

For the ECC dataset I first applied undersampling and feature scaling. Next, I calculated the min_samples parameter for DBScan using the 2xD formula (double the number of features) due to the dataset size. Then, I calculated the optimal epsilon value as 5.5 by using the elbow method. The resulting model creates 1 cluster with 1,392 noise points. Investigating the noise, I found that 407 of the points were positive class instances (29% positive rate). The remaining 85

positive class points were in cluster 1, showing the model correctly flags the majority of the positive class as noise. Investigators could use this model to reliably find fraudulent transactions without the required ground truth needed for supervised learning models. Next, I profiled the noise cluster to determine its characteristics. I trained a decision tree using the cluster identifier as the label. As we can see in Figure 4, the features V1, V8, V14, V16, and V17 are used as decision rules. As found in EDA, V14 and V17 had the strongest correlation with the target, showing these strong correlations helped to separate the target variable.

On the IBM dataset I applied the same preprocessing techniques to undersample and scale the feature set. Next, I calculated the min_samples using 2xD as 86 due to dataset size. Building upon the K-Means analysis, Euclidean distance metric is likely to have issues due to the large count of sparse binary features. I validated this assumption by applying the elbow method and training DBScan with the Euclidean metric. The resulting model created 2 clusters and found 6 noise points. The largest cluster with 55,377 points contained 5,115 of the fraudulent transactions, with the remaining 62 fraudulent transactions in the noise and second cluster, showing DBScan did not separate the fraudulent transactions. Next, I used the Gower distance metric which handles mixed data types. I recalculated the optimal epsilon of 0.025 and trained the model which resulted in 75 unique clusters. As we can see in Figure 5, the clusters separate fraudulent transactions significantly better such as cluster 65, with 0.76% positive rate. We can see in Figure 6 that cluster 65 identifies Saudi Riyal currency payments using digital transactions (ACH). One challenge using this approach is the number of clusters. Without labels an analyst would need to use domain knowledge in combination with the decision tree rules to understand which clusters contain the fraudulent transactions. In summary, this approach shows that the

positive instances can be separated using clustering without labels, providing investigators with a model to uncover anomalies without prior ground truth.

On the PPP dataset I applied the same preprocessing techniques to undersample and scale the dataset. Next, I used a min_samples=10 due to the small positive instance count. Similar to the IBM dataset, Euclidean distance is likely to present issues. To validate this assumption, I baselined DBScan with Euclidean metric and the optimal epsilon and min_samples. The model created 1 cluster and found 1,246 noise points. The largest cluster with 93,849 points contained 92 of the fraudulent loans, with the remaining 3 fraudulent loans present in the main cluster. DBScan did not successfully separate the fraudulent transactions. Next, I used the Gower metric and recalculated the optimal epsilon of 0.03 and trained the model, resulting in 7 unique clusters and 1,503 noise points. As shown in Figure 7, this Gower DBScan separated the positive class better with the points distributed between the second cluster and the noise. Cluster 2 has the highest positive rate with 21% of the points in the positive class. Examining cluster 2 closer using the decision tree approach, I found the cluster represents loans which were not forgiven in underutilized business zones or low approval amounts. In this dataset, DBScan with the Gower metric did not perform as well as the IBM dataset but did improve upon KMeans performance.

Overfitting

I consistently encountered overfitting due to extreme class imbalance when using KNN and Gradient Boosting, where models learned to always predict the negative class. To resolve this I implemented undersampling to reduce the imbalance. Next, I utilized a train test split to validate model performance to ensure generalization to unseen data. Then, I performed five fold stratified cross validation on the training set to determine optimal parameters, using stratification to ensure folds contained both negative and positive class instances and cross validation for

consistency with small fold sample sizes. Additionally, I observed overfitting when using high learning rate in the Gradient Boosting due to trees learning noise rather than underlying patterns.

Metrics and Hyperparameter Tuning

All three datasets included large class imbalances. As a result the accuracy provides an invalid measurement, where 99% accuracy often describes a model that always predicts the negative class. To counter this problem, I used precision, recall, and f1 for model evaluation and selection. Additionally, I used the classification report and examined these metrics across each class to gain a deeper understanding of the tradeoffs between models. For hyperparameter tuning, I used Optuna for optimization and stratified five fold cross validation, which helped to address the small positive instance count in each dataset by ensuring models generalized consistently rather than relying on a single train test split. Stratification was used to prevent folds from containing purely the majority class. To evaluate the impact of hyperparameter tuning, I compared performance of tuned models against baseline models.

Expected and Unexpected Results

An unexpected result in KMeans clustering was the failure to cluster using the Euclidean distance in the IBM and PPP datasets, while the same metric was successful in KNN when using undersampling, standardization, and hyperparameter tuning. In the IBM and PPP datasets, there is a large number of sparse binary variables, making many points seem similar due to their matching zeroes in KMeans. In reality small differences in the features with ones can make points very different, but the Euclidean distance is dominated by the matching features. In KNN we have the labels for training in combination with the distances, allowing a more flexible decision boundary. This allowed us to get strong performance without needing to change the metric. An expected result was the strong performance of Gradient Boosting on each of the

datasets. In the financial crime domain, many techniques are continually created and applied. As companies become aware of strategies and resolve them, criminals continually evolve to beat automated systems. Gradient Boosting, which continually builds trees to predict the error in previous ones, is a strong candidate for this problem statement. While preliminary trees may identify certain patterns in the data, subsequent trees find new patterns the previous missed.

Exploratory Data Analysis

In the KMeans and DBScan clustering techniques I applied domain knowledge acquired through the EDA to determine dimensions of the clusters to explore. In the ECC dataset, this validated my choice of the V12, V14, and V17 features to explore separation of the positive class through plots. Additionally, in DBScan it validated my understanding of the high positive class rate clusters through the decision trees I outlined, where I found the features used for decision rules used features that correlated with the target. In practice, investigators could use these feature insights in tandem with their own domain knowledge to profile clusters. This becomes especially useful in the case that labels are not available. While I was able to know to profile cluster 65 in the IBM dataset and 2 in the PPP dataset due to the prevalence of labels, an investigator would lean on the decision trees to understand the rules that can be used to identify clusters and domain knowledge to make inferences about which clusters contain financial crime.

Figure 1. ECC Learning Rate

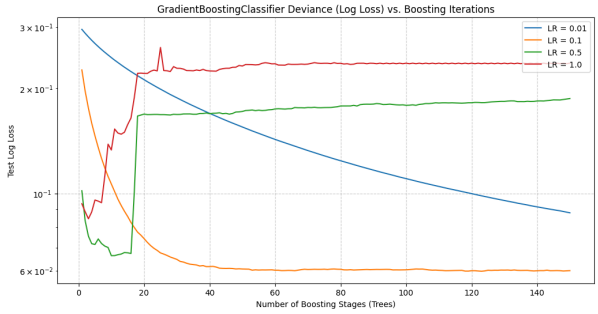

Figure 2. IBM Learning Rate

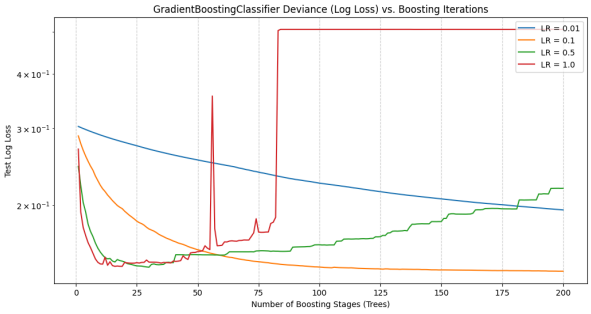

Figure 3. PPP Learning Rate

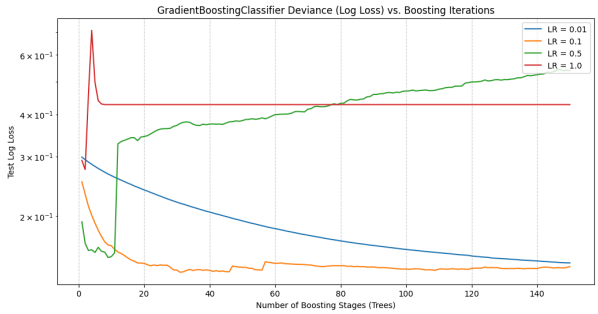

Figure 4. ECC DBScan Positive Class Tree

```
--- DECISION TREE RULES DEFI
|--- V1 <= -2.40
|   |--- V1 <= -3.47
|   |   |--- V14 <= 1.85
|   |   |   |--- class: 1
|   |   |--- V14 >  1.85
|   |   |   |--- class: 1
|   |--- V1 >  -3.47
|   |   |--- V8 <= 0.54
|   |   |   |--- class: 1
|   |   |--- V8 >  0.54
|   |   |   |--- class: 0
|--- V1 >  -2.40
|   |--- V17 <= -1.63
|   |   |--- V16 <= -0.55
|   |   |   |--- class: 1
```

Figure 5. IBM DBScan Clusters

```
--- TOP 10 CLUSTERS BY POSITIVE CLASS RATE ---
 cluster  cluster_size  positive_count  positive_rate
      65          470             361       0.768085
      13         2421            1206       0.498141
       2          347             162       0.466859
      26         3665            1663       0.453752
      45          299             132       0.441472
      54          193              79       0.409326
       5          356             144       0.404494
      61          278             112       0.402878
      29          282             111       0.393617
      46          290             114       0.393103
```

Figure 6. IBM DBScan Positive Class Tree

```
--- DECISION TREE RULES DEFINING CLUSTER 65 ---
|--- Payment Currency_Saudi Riyal <= 3.23
|   |--- class: 0
|--- Payment Currency_Saudi Riyal >  3.23
|   |--- Payment Format_ACH <= 0.81
|   |   |--- class: 0
|   |--- Payment Format_ACH >  0.81
|   |   |--- Receiving Currency_US Dollar <= 0.28
|   |   |   |--- class: 1
|   |   |--- Receiving Currency_US Dollar >  0.28
|   |   |   |--- class: 0
```

Figure 7. PPP DBScan Clusters

```
--- TOP 10 CLUSTERS BY POSITIVE CLASS RATE ---
 cluster  cluster_size  positive_count  positive_rate
       2          110              24       0.218182
      -1         1503              59       0.039255
       3           39               1       0.025641
       1          434               6       0.013825
       0         7400               5       0.000676
       4           69               0       0.000000
       5            9               0       0.000000
       6           31               0       0.000000
```

Figure 8. PPP DBScan Positive Class Tree

```
--- DECISION TREE RULES DEFINING CLUSTER 2 ---
|--- LoanStatusDate_int <= -2.24
|   |--- ForgivenessDate_int <= -1.31
|   |   |--- HubzoneIndicator <= 0.53
|   |   |   |--- class: 1
|   |   |--- HubzoneIndicator >  0.53
|   |   |   |--- class: 0
|   |--- ForgivenessDate_int >  -1.31
|   |   |--- class: 0
|--- LoanStatusDate_int >  -2.24
|   |--- LoanStatusDate_int <= -1.92
|   |   |--- CurrentApprovalAmount <= -0.50
|   |   |   |--- class: 1
```

References

Altman, E. (2025, July 8). *IBM transactions for Anti Money Laundering (AML)*. Kaggle. https://www.kaggle.com/datasets/ealtman2019/ibm-transactions-for-anti-money-laundering-aml

Bukowski, D. (2022, May 25). *Covid PPP loan data with fraud examples*. Kaggle. https://www.kaggle.com/datasets/danb91/covid-ppp-loan-data-with-fraud-examples

Machine Learning Group - ULB. (2018, March 23). *Credit Card Fraud Detection*. Kaggle. https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

AI Appendix

I did not use AI for this submission.